

Clouded Horizons: Planeswalker Chronicles

*Senior Project – Benji James Powell
August 14, 2008 to February 27, 2010*

Contents

CLOUDED HORIZONS: PLANESWALKER CHRONICLES.....	1
PREFACE.....	3
CHAPTER I: INTRODUCTION	4
1.1: WHAT IS A ROLE-PLAYING GAME?.....	4
1.2: WHY CREATE AN RPG?	4
1.3: WHAT IS XNA GAME STUDIO?.....	4
1.4: WHAT IS TILE STUDIO?.....	5
CHAPTER II: THE PROJECT	5
2.1: XNA'S UNDERLYING CODE SEGMENTS	5
2.2: <i>CHARACTER, MONSTER, AND NPC</i> CLASSES	7
2.3: <i>SPRITE AND TILE</i> CLASSES.....	8
2.4: <i>MENUBUTTON</i> CLASS	9
2.5: <i>JOB AND SKILL</i> CLASSES	10
2.6: <i>ITEM, WEAPON, AND ARMOR</i> CLASSES.....	10
2.7: <i>ENCHANTMENT AND EFFECT</i> CLASSES.....	11
2.8: <i>QUEST</i> CLASS	11
2.9: <i>CONSTANTS.CS</i>	12
2.10: <i>GAME1.CS</i>	13
2.10.1: <i>Enumerated Cases and Variables</i>	13
2.10.2: <i>General Use Methods</i>	15
2.10.3: <i>Combat System Methods</i>	18
2.10.4: <i>Game1.Update()</i>	20
2.10.5: <i>Game1.Draw()</i>	21
CHAPTER III: CONCLUSION	22
3.1: LESSONS	22
3.2: WHAT COULD BE DONE DIFFERENTLY?	22
3.3: WHAT COULD BE ADDED?.....	23
3.4: KNOWN BUGS OR GLITCHES	23
3.5: ACKNOWLEDGMENTS.....	23

Preface

This project, *Clouded Horizons: Planeswalker Chronicles*, was designed for my senior project course at Charleston Southern University. The basic idea is that of a console role-playing game with an original storyline that pits the player against the forces of an ancient being. I began developing this project in August of 2008 with a storyline and system already in mind, and while the entirety of the storyline is far too vast for me to show in this project, this will give a glimpse into the system that I have envisioned and created so far. The project was created using several different pieces of software, including:

- Microsoft Visual Studio 2005
- XNA Game Studio 2.0 add-on for Visual Studio
- Microsoft Cross-Platform Audio Creation Tool (XACT)
- Tile Studio
- Macromedia Fireworks

Chapter I: Introduction

1.1: What is a role-playing game?

Wikipedia's article on the subject gives the following definition:

*"A role-playing game (RPG, often roleplaying game) is a game in which the participants assume the roles of fictional characters."*¹

The basic idea is that a role-playing game (or simply RPG, as I'll refer to it from this point forth) is a game in which the player assumes the roles of one or more fictional characters within a story and progresses through a storyline of some sort. As the game progresses, the characters accumulate wealth both in terms of the monetary system of the game world as well as usable items and equipment, gaining strength and defeating various enemies within the world. The purpose behind RPGs is a very simplistic one from a player's standpoint – acquire wealth, defeat stronger enemies, advance the storyline, and repeat until finished. Most RPGs follow this simplistic formula, with the only differences between them being various implementation choices such as art style, combat system, and methods of storyline advancement.

1.2: Why create an RPG?

This project was designed to allow me a better idea of how a game such as this might be created, as well as give a greater understanding of large projects and the steps needed to create and maintain one. It was designed to be both a strong challenge and a good learning experience at the same time, without being too complex to understand or complete. This was my first experience with projects of this size and scope, and a large amount of time was spent simply learning how to handle such a large and complex project, how to effectively debug code for such a project, as well as learning the software involved, specifically XNA Game Studio and its associated language, a derivative of the C# programming language with special features added in for game development.

1.3: What is XNA Game Studio?

XNA Game Studio 2.0 (or XNA) is an add-on for the Microsoft Visual Studio suite that was designed to allow programmers to write code for Windows operating system-based games, as well as games for the Xbox 360 video game console. It installs directly on top of an existing Visual Studio install, integrating itself into the program similarly to any other add-on for the program. The XNA suite features its own programming language, which is based on the C# language as stated above, and has additional features specifically for game developers such as added support for music through the Microsoft Cross-Platform Audio Creation Tool, or XACT, and an underlying segment of code in every created project which handles the initialization of the game loop and ensuring that it executes, which is hidden from the user and cannot be opened or modified. This allows programmers to focus on game-specific code without having

to create an entire executing base from scratch before they can even begin programming the game itself. In this way, Microsoft hoped to open up and simplify the game-creating experience to programmers who may not otherwise have had the experience to do so.

1.4: What is Tile Studio?

Tile Studio is a software program that allows the user to create small images, or tiles, which may then be placed on maps and exported together or on their own. This project utilized a tile-based approach to environment creation, focusing on creating arrays of tiles that the characters would then navigate as the game world. Tiles are manually drawn within the program, similar to any drawing tool such as Paint, and can be created in any size desired; the tile maps are created similarly to two-dimensional arrays, having a height and width based on a number of tiles instead of a size in pixels.

Chapter II: The Project

Now that the nature of the programs involved has been explained, the project itself can now be explained in earnest.

2.1: XNA's Underlying Code Segments

It was stated above that XNA Game studio provides the programmer with a small segment of code when a new project is created. XNA provides the programmer with two starting class files: Program.cs and Game1.cs. Game1.cs contains a total of five empty methods within its class header, which are as follows.

```
protected override void Initialize() { base.Initialize(); }

protected override void LoadContent() { }

protected override void UnloadContent() { }

protected override void Update(GameTime gameTime) { base.Update(gameTime); }

protected override void Draw(GameTime gameTime) { base.Draw(gameTime); }
```

These five methods provide the framework for the underlying system that allows an XNA project to run as a game.

The `Initialize()` method is used to initialize all variables, classes, or any other data that is needed before the game actually begins running, such as required services. One exception is image data, which is handled by the `LoadContent()` method. Any variables or method calls relating to loading game data should be placed here; for this project, that meant method calls relating to loading in default map, quest, character and enemy data from text files, giving the player starting inventory items, changing the screen resolution, and initializing the game's audio content, among other things.

The `LoadContent()` method handles the only things that the `Initialize()` method did not – data which must pass through the Content Manager before use. XNA handles certain things, such as image and audio content, through the use of the *ContentManager* class, which handles the importing of image and audio files and their conversion into files that the XNA framework can work with and display. This class contains a `LoadContent()` method also, which must be called on images before they can be utilized within the system. It also handles the loading of specific fonts, which use the *SpriteFont* class, into the system. By default, XNA provides the programmer with an unmodifiable variable of type *ContentManager*, *Game.Content*. For example, the code segment below will load a custom font style from a file with the ".spritefont" extension into a *SpriteFont* variable named *lineFont* for use within the project.

```
lineFont = Content.Load<SpriteFont>("LineFont");
```

The `UnloadContent()` method is used to unload any non-Content Manager content from the game, releasing its resources back into memory. This function was left empty for purposes of this project, as the destruction of non-Content Manager objects was handled within other methods when required so as to better time their removal.

The `Update()` method serves as one of the two main functions that keep the game running, the other being the `Draw()` method. In the `Update()` method, programmers can place all of the game logic code that is needed for their specific game, which the XNA project will run automatically for them. Both the `Update()` and `Draw()` methods serve as a sort of underlying, constant loop for the program, which both make their best effort to run 60 times per second. If the combined time for the two methods is less than 1/60 of a second, the program will simply wait for the next call to `Update()`. The *gameTime* variable works with both the `Update()` and `Draw()` methods as a snapshot of the current time, expressed in milliseconds. A large amount of the code present in this project resides in `Update()` due to the amounts of game logic present in an RPG to handle quests, character interactions, and combat.

The `Draw()` method is the other main function that assists the game with running smoothly. Here, all of the image objects needed by the game can be drawn, including both images and fonts. As with `Update()`, the `Draw()` function constantly runs behind the scenes, refreshing and redrawing objects as needed. Before any drawing to the screen can be done, however, the programmer must first have both a *GraphicsDeviceManager* object and a *SpriteBatch* object. The *GraphicsDeviceManager* object handles the configuration and management of the graphics device for the game itself, allowing the user to use functions to do things such as clear the screen completely in preparation for redrawing. The *SpriteBatch* object is the object which actually allows the user to draw images, holding such methods as `Draw()` and `DrawString()`. Before drawing an image with the *SpriteBatch* object, the *SpriteBatch.Begin()* method must be called to prepare the object for drawing. Similarly, when finished drawing, the *SpriteBatch.End()* method must also be called before `Begin()` may be called again.

The `Program.cs` file contains the other half of the underlying system code, with only one function present within it, shown below.

```
static class Program
{
    /// <summary>
    /// The main entry point for the application.
    /// </summary>
    static void Main(string[] args)
    {
        using (Game1 game = new Game1())
        {
            game.Run();
        }
    }
}
```

The *game.Run()* method is the method which keeps the `Update()` and `Draw()` game loops running; however, the programmer cannot modify or even see the code for the method in any form, making it impossible to accidentally modify or damage and therefore harm the game's operation.

2.2: Character, Monster, and NPC Classes

The *Character* class is the basis of the entire game; without characters, there would be no game at all. While the class itself is quite long (around 800 lines), the majority of the code within is not complicated. The class holds a large amount of variables that contain all the character's information, such as the character's name, image object, position on the map, experience values, and other attributes for use in combat. Along with functions to access and modify these values, the class also contains other general

functions – functions to calculate character statistics given certain values, functions that allow the character to equip or remove weapons and armor, as well as functions to allow the character to move and redraw them on the map, such as the following:

```
public void MoveUp(SpriteBatch spriteBatch, Tile[,] map)
{
    charDirection = Direction.Up;

    // If a valid move...
    if (charXPos >= 0 && charYPos - 1 >= 0)
    {
        // If the tile is walkable...
        if (map[charXPos, charYPos - 1].isWalkable())
            Draw(spriteBatch, map, charXPos, charYPos - 1);
        else
            Draw(spriteBatch, map, charXPos, charYPos);
    }
    else
        Draw(spriteBatch, map, charXPos, charYPos);
}
```

This function first tests that the tile that the character wants to move to is actually still a part of the map; if it isn't, it simply redraws the character in place. If it is a part of the map, it then checks to see if the tile itself can be walked upon. If it can, it moves the character to that tile and redraws, and simply redraws in place if not. Three other functions handle the same logic for the other three directions, Down, Left, and Right. The class also inherits from another class, *IComparable*, which allows for the use of a comparing function to determine the order that characters will act in combat.

```
public int CompareTo(object r)
{
    Character right = r as Character;
    if (getAgility() > right.getAgility()) return 1;
    else return 0;
}
```

Characters with higher Agility scores will act before ones with lower scores, and these comparisons are restarted after all characters in a battle have moved.

The *Monster* class inherits from the *Character* class, giving it all of the same functions that a *Character* has even though some functions are not used, such as the movement functions. However, the *Monster* class also contains two special functions that are used to access or modify the amount of experience earned after defeating them.

Similarly, the *NPC* class also derives from the *Character* class, but contains three functions that the above two do not have; these functions return whether the NPC is a shopkeeper or not, the default dialogue that the NPC possesses, and the name of the map that the NPC appears on.

2.3: *Sprite* and *Tile* Classes

The *Sprite* class was created as a way to abstract away some of the more confusing pieces of using the *ContentManager* class to load and display images. The *Sprite* class has only three variables: an image name of type *string*, a texture image of type *Texture2D*, and a position variable of type *Vector2*. Along with these variables are the usual functions to get and set their values, along with two other functions that handle the abstraction of the Content Manager.

```
public void LoadContent(ContentManager cm, string name)
{
    spriteTexture = cm.Load<Texture2D>(name);
}

public void Draw(SpriteBatch sprite)
{
    sprite.Draw(spriteTexture, Position, Color.White);
}
```

The `LoadContent()` method is used in the *Game1* class' `LoadContent()` method in order to load in a given *Sprite*'s image through the Content Pipeline, and serves as simply an easier-to-remember way for the programmer to load an image. Likewise, the `Draw()` function serves the purpose of the *SpriteBatch* object's `Draw()` function, but in a more succinct manner.

The *Tile* class is similar to the *Sprite* class in that it provides a more neatly-organized setup for some of the basic functions of the game map. It holds only five variables: a variable to store the upper-left most pixel positions of the tile for use in drawing characters upon the tile, stored as a *Vector2*, two *Boolean* variables that return whether the tile in question is a door that leads to another location as well as whether the tile is able to be walked upon, a *string* variable that holds the name of the destination map for tiles which are doors, and a *Sprite* class variable that stores the tile's image. It contains functions to change all of these values, but the drawing functions are handled via the *Sprite* class' drawing functions.

2.4: *MenuButton* Class

The *MenuButton* class serves as the basis for all menu interaction within the project; all menu screens rely on a mouse-based interface for navigation. The class holds several logical functions that determine whether the mouse is over a given button and whether it has been pressed or not, as well as other functions to update the appearance of a button and draw the button itself. It also contains a *string* variable that holds the text displayed on a given button and a *Sprite* class variable for the button's image.

```
public void MouseIsOver()
{
    MouseState m = Mouse.GetState();
    if ((m.X >= image.Position.X && m.X <= image.Position.X + 145) &&
        (m.Y >= image.Position.Y && m.Y <= image.Position.Y + 34))
        over = true;
    else over = false;
}
```

```

    }

    public void IsLeftClicked()
    {
        MouseState m = Mouse.GetState();
        if (over == true && m.LeftButton == ButtonState.Pressed)
        {
            pressed = true;
        }

        if (over == true && m.LeftButton == ButtonState.Released &&
pressed)
        {
            lClick = true;
            pressed = false;
        }
    }
}

```

The first function serves only to determine whether the mouse is being hovered over or not. If the mouse's current location (held internally as a *Vector2* object) is within the boundaries of the *MenuButton*'s image, then it sets the variable *over* to true. This, in turn, allows the second method to reliably determine when a button is clicked. The *IsLeftClicked()* method first checks to see if the button is over and the mouse's left button has been pressed, at which time it sets the variable *pressed* to true. After that, if the mouse is both over the button and the button has been released, provided that the button was clicked beforehand to set *pressed* to true, it returns a valid left click.

The class' *Update()* function simply calls the above two functions in order to update the current status of the button. This function is called within the *Game1* class' *Update()* function when needed, so that the button statuses are continually up-to-date. *Draw()* works similarly to the function of the same name in the *Sprite* class, but also includes logic to change the color of the button's text to signify that the button is being hovered over. There is also a *LoadContent()* method which works identically to the one mentioned above in the *Sprite* class.

2.5: *Job* and *Skill* Classes

The *Job* class serves as one of the key features of a character, giving them a specific role within the player's party and different abilities depending on the Job chosen (Job was chosen as a name to replace Class, for obvious reasons). The *Job* class holds several variables that are used to modify a character's stats based on the selected Job, as well as variables that represent the accumulated experience in that Job, the current level of that Job, and an array for skills belonging to that Job. Much like the *Character* class, it is lengthy but not overly complex in design. Along with the functions to get and set the values of the statistic variables, it also contains two functions that allow the adding of skills to the array, limited by the size of the containing array, as well as a function that checks and updates the character's list of skills, which is used whenever the character's Job is first created or when the character gains a level in that Job.

The *Skill* class is as essential to the *Job* class as a *Job* is to a given character; without the *Skill* class, the only difference between characters would effectively be different statistics. It, too, has several different variables, including variables for the skill's name and description, how much it costs the character in terms of energy to use the skill, how much damage it does, what type of skill it is, and who it hits in battle. All of these variables have functions to retrieve and modify their values, and the *Skill* class also contains a lengthy function that reads in a *Skill* from a file, which is used when *Monster* data is read in from a file to give them skills without the need to tie them to a specific *Job*.

2.6: *Item, Weapon, and Armor* Classes

These three classes go hand-in-hand with one another, with the latter two deriving from the first. The *Item* class is another fundamental class for an RPG, as without items, the game would be significantly less varied with regards to character abilities. The class contains variables for such things as the item's name and description, the quantity of that item, and the price, as well as methods for modifying all of the above. Like *Character*, it also extends the *IComparable* class, with a comparison method for determining if two items have the same name in order to stack them in the same inventory slot.

```
public int CompareTo(object r)
{
    Item right = r as Item;
    if (this.itemName == right.getItemName()) return 0;
    else return 1;
}
```

In addition to the methods listed above, it also contains an additional method:

```
public virtual void use(Character tgt) { }
```

This method is passed down to many of its child classes, including *Weapon*, *Armor*, and several other classes which will be listed in section 2.9, in order to allow them specific logic for how they are used, as some items such as healing potions are consumed after use, while to “use” a weapon may simply mean to equip the weapon *for* use.

The *Weapon* and *Armor* classes are similar in their setup – both simply contain variables relevant to that particular type of item, such as a damage value, the type of weapon, and any associated elemental force for weapons, and defense values and the type of armor for armor-class items. Both classes also contain a variable of type *List<Enchantment>* for storing a list of magical enchantments that either of these item classes may have; the *Enchantment* class is described in detail below.

2.7: *Enchantment and Effect* Classes

Equipment that characters find in the world of an RPG is often imbued with powerful magical enchantments, which is what the first of these two classes represents. The *Enchantment* class contains variables for bonuses that could be granted to each of a character's statistics, such as their strength or defense against physical attack, as well as functions to modify each of their values. It has one other function, `determineEffect(string statBoosted, double amount)`, which uses a *switch*

statement based on the *statBoosted* variable to determine which variable receives the amount of the increase.

The *Effect* class, by contrast, serves as the base class for status effects which can benefit or hinder a character in battle. It has only a few variables of its own, those being the type of status effect (positive, negative, or healing), the name of the effect, how many turns it lasts in battle, and whether it has been applied or not during that turn. It also has three special functions that are passed down to the child classes:

```
public virtual void apply(Character tgt) { }
```

This function handles the logic when a status effect is applied to a target, be it positive or negative. Each status effect does different things, from poisoning the target to lowering a statistic for several turns.

```
public virtual int applyHeal(Character tgt) { return 0; }
```

This function handles the logic for healing-type effects, such as curative magic. It returns the amount that the target is healed, which is used for the damage animation within the combat logic, covered in section 2.10.

```
public virtual void remove() { }
```

This function handles the logic when a status effect is removed from a target; this may involve simply removing the effect from the character's list of status effects, or also restoring a stat that was raised or lowered by the effect.

2.8: Quest Class

The *Quest* class is another major component of an RPG – without quests, and by extension a storyline, the game becomes nothing more than a bunch of numbers and pictures. This class serves as a parent for individual quest objects, holding basic data like the quest's name, whether it is completed or not, and variables for representing where the character is in terms of quest progress. It has several functions besides those that modify variables, including one that reads a quest from file, and another that modifies an NPC's dialogue if it has relation to the quest, shown below:

```
public void modifyDialogue()
{
    // If the character being spoken to has dialogue that pertains to the
    // quest...
    if (Game1.activeQuest.dialogueTable.ContainsKey(Game1.speakingNPC
        .getCharacterName()))
    {
        // Assign their dialogue entry the dialogue based on what step of
        // the quest they are at.
        string[] dialogue = (string[]) (dialogueTable[Game1.speakingNPC.
            getCharacterName()]);
        Game1.NPCDialogue = dialogue[Game1.activeQuest.getCurrentStep()];
    }
    else
```

```

    {
        // Leave their dialogue as the standard dialogue.
        ArrayList NPCs = (ArrayList)Game1.npcTable[Game1.mapName];

        // Find the correct NPC in the list.
        foreach (NPC val in NPCs)
        {
            if (Game1.speakingNPC.getCharacterName() ==
                val.getCharacterName())
            {
                Game1.NPCDialogue = val.getDialogue();
            }
        }
    }
}

```

The function first looks in a *Hashtable* object named *dialogueTable* belonging to the currently active quest object, searching for a key with the name of the NPC being spoken to. If it finds it, then it retrieves the list of quest dialogue for that NPC from that table, setting the *NPCDialogue* variable to the correct line of dialogue for the current quest progress. If it does not find that the NPC in question has dialogue, it defaults back to the NPC's standard dialogue.

Along with this function, there are two other functions, `public virtual void checkProgress()` and `public virtual void giveReward()`, which are passed down to the child classes of the *Quest* class in order to handle quest-specific logic, such as advancing the quest and determining when it is completed.

2.9: *Constants.cs*

The *Constants.cs* file is not a class in itself, but is rather a repository for groups of similar classes used for organizational purposes. Among these classes are derived classes for the *Job*, *Skill*, *Effect*, *Item*, and *Quest* classes.

The derived classes for the *Job* class represent specific character Jobs available within game, such as an Alchemist or a Warrior. Each of these classes has only a constructor with one parameter, the level of the Job, which calls the base *Job* constructor with a specific list of values and the aforementioned level in order to modify character statistics accordingly.

There is only one derived class for the *Skill* class, a class named *Limit*. The class represents special, character-specific techniques that are more powerful than the standard class techniques, but are also fewer in number and more difficult and expensive to perform. It has a single class-specific variable, *energyCost*, which represents the amount of Energy (a specific value that every character has) to perform the technique.

The derived classes for the *Effect* class include three different types of classes. First, they include the different status effects which can afflict or aid a character, such as *Poison* or *Weakness*. Each of these classes include a constructor that calls the base *Effect* constructor with a specific duration, as well as setting the appropriate type of status and status name. They also implement the *apply(Character tgt)*

function from the parent class, and those which have effects which require restoring when removed also implement the *remove()* function. Depending on the class, some have variables to hold the old, unmodified stats for restoring upon removal. Second, they include similar classes for effects given from skills, and work much the same as the status effect classes. Third, they include similar classes for healing skill effects, which all implement the *applyHeal(Character tgt)* method from the *Effect* class.

The derived classes for the *Item* class include various non-equipment items meant for restoring health or energy to a character. Each of these classes implement the *use(Character tgt)* method within the *Item* class for their use logic.

Finally, the derived classes for the *Quest* class include classes for each specific quest in the game. Each of these classes have a constructor which calls the *Quest* class constructor with a specific name, used for reading the quest data through the *readQuestFromFile(string fileName)* function, as well as implement the *checkProgress()* and *giveReward()* functions from the parent class for quest-specific logic.

2.10: *Game1.cs*

The *Game1.cs* file is the largest file in the program, and rightfully so, as it contains all of the game logic that makes the game run as it should. It holds a large number of different enumerated cases, variables, and methods that are used within the five methods listed in section 2.1. The variables are subdivided into several different sections, based on their function, while the methods are subdivided into two sections – methods for general use, and methods for use within the combat system.

2.10.1: Enumerated Cases and Variables

The enumerated cases within the *Game1.cs* file are not strictly necessary, but help to see specifically what logic is taking place, especially within the combat functions. Enumerated types exist for the type of weapon or armor, be it a Light, Medium, or Heavy armor or a Melee, Magic, or Ranged weapon, as well as character facing direction (Up, Down, Left, or Right), elemental types, skill types, status effect types, skill targets, and states for both menus and the game itself, which will be discussed further below.

The variables section, as mentioned above, is subdivided into several different sections for organizational purposes. Among these are audio variables, combat system variables, game state variables, graphics variables, map variables, menu variables, quest variables, variables for the player character and their inventory, lists for allies, enemies, and NPCs, lists of monsters in specific areas, and font variables.

- Audio variables handle the setup and storage of the audio files within the program as well as the audio equivalent to the Content Manager, the *AudioEngine* class. To play audio, an object of this type must first be instantiated, along with two other objects – a *SoundBank* and a *WaveBank* object. One of each of these types exist within this section, as well as several other variables of type *Cue* which hold the sounds, as well as methods for playing, stopping, or pausing them. Also in this section are a number of *string* variables that hold specific song names

for play at certain times, such as themes for moving around in the overworld map or combat themes.

- There are a large variety of combat variables, all of which serve very specific purposes within the combat system. *Boolean* variables exist for showing whether or not a skill's cost has been paid, whether or not the animation showing damage to a character should be played, whether experience has been given for the battle, as well as for whether a specific type of skill was used. Two *Character* variables exist, denoting the active character (the character currently acting in the battle), as well as the target of a single-target skill. Two *Character* arrays also exist, one for the initiative order (the order in which characters act during battle), and another for the list of targetable characters. A single *Color* variable denotes the color that the damage animation should be, depending on whether or not damage or healing was done. Several *integer* variables exist to hold the menu choices taken in battle, the remaining counts of enemies, allies, and characters in the initiative order for the round, the indices of the skill or *Limit* technique used, and the total experience awarded from the battle. *Integer* arrays also exist for the experience tables, denoting when a character gains a level. *String* variables exist for the amount of damage or healing done for use in animation, the skill name for displaying on screen, and *string* arrays exist for displaying messages after battle as well as holding damage or healing amounts for multiple-target skills.
- Overworld variables serve purposes for things such as dialogue and checking if something exists when the action button, *Space Bar*, is pressed. *Boolean* variables exist to check whether dialogue has ended between the player and an NPC, whether dialogue has been parsed to fit the message box before displaying, and whether an object that the player has pressed the action button on is an NPC, a treasure chest, or a boss encounter.
- Only three game state variables exist, but they are among the most important in the project. A single variable of type *Stack<GameState>*, *states*, holds a stack of the current states of the game, such as being on the overworld map or being in a dialogue screen. A single *GameState* variable, *currentState*, holds the topmost member of the aforementioned stack, and a *MenuState* variable, *menuState*, holds the current menu state for combat.
- Like the game state variables, some of the graphics variables are extremely important. Both a *GraphicsDeviceManager* and *SpriteBatch* object exist, without which drawing would be impossible, and several more *Sprite* variables exist for the player character's appearance on the world map, as well as the title screen background and a selection arrow that denotes the active character in combat.
- Again, the map variables are very important to this project. A two-dimensional array of the *Tile* type holds the current map of tiles, with another of the same holding any objects that are to be placed on the map. A final two-dimensional array of the *Item* type represents items present on the map, whether in treasure chests or hidden on the overworld. A single *Boolean* variable determines whether a random encounter can be started on the current map, and a *string* variable holds the current map name.
- The menu variables hold the components of both combat and non-combat menus as *MenuButton* arrays. Such arrays exist for all such menus, and *MenuButton* objects exist to

represent the individual buttons in such menus. Several *integer* variables also exist, holding things such as the default positions of buttons (in pixels) and the choices made in a menu screen.

- Quest variables are few in number, but pertain to another important part of the game. One *Boolean*, *integer*, and *string* variable each exist, holding whether a quest reward is present, the number of quests completed, and dialogue for quest rewards, respectively, and a *Quest* class variable exists that denotes the currently active quest.
- Variables for the player character include such things as the player character themselves, represented by a *Character* variable, their inventory, a *List<Item>* variable, and *integer* and *string* variables for holding the current amount of money they have and the name of that money.
- Font variables are all of the type *SpriteFont*, and hold the individual font styles read from the file of the same name.
- Several array variables exist for lists of defined length, such as the parties of the player and enemies (during combat), names of NPCs, enemies in a particular area, and the number of bosses.
- Two important *Hashtable* objects also exist – one for the list of NPCs in the game, where the key is the area name, and the other for a list of items in the game where the key is the item name.

Along with these variables, three other important variables exist, two of type *KeyboardState* and one of type *MouseState*. These variables hold the state of keys on the keyboard and mouse for use in the *Update()* function.

2.10.2: General Use Methods

The general use methods serve a variety of purposes within the project, including but not limited to setting up audio input, reading data from files, modifying game states, modifying inventory, setting up menus graphically, determining whether a specified character is friend or foe, and more.

The audio themes are relatively simple and straightforward, as shown by the following:

```
public void initializeAudio()
{
    audioEngine = new AudioEngine("Content\\CHAUDIO.xgs");
    soundBank = new SoundBank(audioEngine, "Content\\Sound
    Bank.xsb");
    waveBank = new WaveBank(audioEngine, "Content\\Wave Bank.xwb");

    loadTheme(ref titleTheme, titleThemeName);
    loadTheme(ref overworldTheme, overworldThemeName);
    loadTheme(ref battleTheme, battleThemeName);
    loadTheme(ref gameOverTheme, gameOverThemeName);
}

public void loadTheme(ref Cue theme, string name)
{
    theme = soundBank.GetCue(name);
}
```



```
}
```

These two functions handle the loading of all the songs within the project, from the title theme to combat themes. The Audio Engine, Sound Bank, and Wave Bank all come from an XACT project in the project folder. Before a sound can be used in XNA, it must first be imported into an XACT project and placed in the sound and wave banks, where it is given a cue that can be used to start and stop it in XNA. Once the XACT project is set up, the three aforementioned variables can be initialized, which this function does, and the themes can be retrieved using their cues – in this case, the song name itself. Functions for the playing and stopping of themes also exist, such as the following:

```
public void playOverworldTheme()
{
    if (overworldTheme.IsPlaying == false)
    {
        if (overworldTheme.IsPrepared)
            overworldTheme.Play();
        else if (overworldTheme.IsPaused)
        {
            overworldTheme.Resume();
        }
    }
}
```

This is among the more complicated of these functions, which is still rather simple. If the theme isn't playing, then check to see if it is ready to play. If it is, begin playing it, and if not, check to see if it's paused. If it is, then resume it. This comes into play during things such as the transition from combat back to the overworld, where the overworld theme would be paused to allow the battle theme to play.

The file reading functions do exactly that – read various kinds of input from files. These include encounter tables for monsters within an area, lists of various kinds of items, character data, and NPCs. In the case of NPCs and items, once they are read in, they are inserted into their appropriate *Hashtable* object and stored for later use. The encounter tables are stored in normal arrays, as is the character data for the player's party.

Only two game state functions exist, and they are extremely simplistic:

```
public void changeState(GameState state) { states.Push(state); }

public void exitCurrentState() { if (states.Count > 1) states.Pop(); }
```

When changing from one state to another, the *changeState* function is called, and *exitCurrentState()* is called when leaving that state.

The inventory functions are four in number, with functions for adding and removing items as well as returning the inventory itself as a list or the number of item slots taken in the inventory.

```
public void addItem(Item target)
{
    if (inventory == null)
    {
```

```

        inventory = new List<Item>(1);
        inventory.Add(target);
    }
    else
    {
        int itemIndex = -1;
        for (int i = 0; i < inventory.Count; i++)
        {
            if (inventory[i].getItemName() == target.getItemName())
            {
                itemIndex = i;
                break;
            }
        }

        if (itemIndex >= 0)
            inventory[itemIndex].setQuantity(inventory[itemIndex].getQuantity() + 1);
        else inventory.Add(target);
    }
}

```

The add function is the most complex of these, due to taking into account whether the inventory exists or not at the time of creation as well as checking to see if the item exists, then incrementing the quantity instead of simply adding the item as a duplicate.

The menu creation functions are similar to the file reading functions, doing exactly what the name implies. Most of them are fairly small, serving to insert *MenuButtons* into their proper places and make sure they will display on the screen properly once that is done.

Many of the other general functions are both simple and made to serve a very specific purpose, and were often done to clean up the code somewhat and make the purpose of certain sections more obvious. For example, the following function changes the screen resolution to a desired amount:

```

public void changeResolution(int width, int height)
{
    graphics.PreferredBackBufferWidth = width;
    graphics.PreferredBackBufferHeight = height;
    graphics.ApplyChanges();
}

```

The fact that those two variables are the ones that need to be changed to affect the screen resolution may not be immediately evident to someone looking at the project's code, and so the function was created in order to make the purpose more obvious and intuitive. Other functions were made as a workaround to something that would have been far too complex and unnecessary otherwise, such as this function to draw horizontal lines on the screen:

```

public void DrawHLine(int startX, int endX, int Y, int increment, Color c)
{
    for (int i = startX; i <= endX; )
    {
        spriteBatch.DrawString(lineFont, "-", new Vector2(i, Y), c);
        i += increment;
    }
}

```

```
i += increment;
```

Instead of using images of horizontal lines to draw them on the screen, this allows the placement of lines of any length at any place on screen, which reduces the need for even more graphical objects.

Several other functions serve to set up the entirety of the world which the player will explore and interact with. Three functions – *loadMap()*, *loadMapObjs()*, and *loadMapItems()* – work together to create the two-dimensional arrays for the map and its associated decorations and items. Each read files in the format [map name].txt, [map name]Objs.txt, or [map name]Items.txt, and proceed to fill the *map*, *mapObjs*, and *mapItems* variables with the correct tiles and objects for the map. Once that is finished, the *drawMap()* and *drawPlayer()* methods can take over, drawing and redrawing the map and player as positions and tiles change.

2.10.3: Combat System Methods

The combat system is an essential part of any RPG – storyline is important to a game, but there is only so much interactivity in scripted events and actions. The combat system for this project is fairly involved, somewhat conceptually and more so mechanically, and the functions are rather large and often complicated, if not particularly complex in terms of coding ability. A few of the major functions are described below.

The *checkInitiative()* function runs at the beginning of every turn, and serves to keep track of the current battle order. It removes the character that acted in the previous turn, and sets the new active player to the next spot in the initiative array. If that array is empty, then it recalculates the order of initiative and refills the array for the next round.

The *fight()* function is the most basic, and yet one of the most complex functions in the combat system. It takes two parameters, an *attacker* and *defender* both of type *Character*, and handles the Fight command on the battle menu. It can be broken down as such:

- Use a switch statement to determine what kind of weapon the attacker is holding – Melee, Magic, or Ranged – then retrieve the base damage of that type that the character deals.
- Create an offset of 20% of that damage to vary the actual damage amount. Once that is complete, find the minimum and maximum damage that may be dealt by using these two values, and randomly select a number in that range.
- Check to see if the attack is a critical hit (for double the damage) by randomly rolling a number between 0 and 1. If the number is between 0 and the attacker's chance to critically hit, the attack does so.
- Use a switch statement to determine what kind of defense – physical or magical – will be applied to the attack, then reduce the damage by a random percentage of that amount. If the damage is less than 1, it becomes 1.

- Enter the resulting damage (and a Critical Hit note, if applicable) into the *dmgVals* array for outputting in the animation.
- Give the attacker a random amount of Energy between 2 and 4.
- Reduce the defender's Stamina by the amount of damage dealt.

The *useSkill()* function has two overloads – one to deal with multiple-target attacks, and one for single-target attacks. Like the *fight()* function, it is long and involved, and can be broken down similarly to that function:

- Check to see whether the skill used is a main class skill, a subclass skill, or a *Limit* technique. This determines which array within the character's object to go to.
- Determine whether the skill is an offensive, defensive, or healing skill.
- If the skill is offensive, proceed as with the *fight()* function. However, award *Energy* based on the strength of the skill performed.
- If the skill is defensive, apply any effects to the user.
- If the skill heals or restores Mana, apply the effect and place the amount of damage or Mana healed in the *dmgVals* array for outputting in the animation.

The multiple-target function uses the single-target version, applying it to all targets in the array, as follows:

```
public void useSkill(int index, Character attacker, Character[] targets)
{
    costPaid = false;
    for (int i = 0; i < targets.GetLength(0); i++)
    {
        if (targets[i] != null && targets[i].getCurStamina() > 0)
        {
            useSkill(index, attacker, targets[i]);
            dmgVals[i] = dmgStr;
            posVals[i].X = targets[i].image.getX() +
                targets[i].image.getWidth() + 25;
            posVals[i].Y = targets[i].image.getY() + 25;
        }
        else break;
    }
}
```

The *AITurn()* function deals with the actions undertaken when the computer player (CPU) takes a turn in combat. It uses randomly-generated numbers to determine the CPU's next action, and then acts accordingly, calling either the *fight()* or *useSkill()* methods as appropriate.

The *showSkillInfo()* and *showLimitInfo()* functions serve to display information about the skill just above the combat menu when a skill's button is hovered over. They are each called within *Game1's* *Draw()* function when their menu is displayed.

The *endTurn()* function handles all logic that needs to happen at the end of a turn, such as applying the effects of any status effects that may be afflicting a character, adding to the total experience to be given at the end of the fight for a defeated enemy, removing defeated enemies or allies, and setting up the target array for the next turn.

The *randomEncounter()* function handles the random monster encounters that the players will have while walking through parts of the overworld, and is called every time the player moves. It first uses random numbers to determine if there will be an encounter; if an encounter is chosen, it then determines the number of enemies that will appear. Once that is chosen, a switch statement looks to the map name to determine which encounter table to look through, and chooses one random monster from the table for each enemy present. It then sets up the music and enemy images for the battle, before changing to the pre-battle state to initiate the encounter.

2.10.4: Game1.Update()

The *Update()* function contains roughly half of the game logic that makes the game perform, and as such is one of the two longest functions in the program. Several different *switch* statements were used within to make sure that the correct code triggered at the correct time, depending on the *currentState* and *menuState* variables. Some functions and code within the method do not rely on the value of those variables, such as the following:

```
// Get the current keyboard/mouse state (which keys are currently pressed and released).
newState = Keyboard.GetState();
mouseState = Mouse.GetState();

// Get the current game state.
currentState = states.Peek();

// Exit if ESC pressed.
if (newState.IsKeyDown(Keys.Escape)) this.Exit();
```

However, they are a clear minority within the *Update()* method, as they are among the only segments of code that do not rely on the current state. The main body of the *Update()* method comes from a single *switch* statement on the *currentState* variable, which then checks the value of that variable against one of fifteen different cases, such as being at the title screen of the game or being inside a shop menu. A sample case is listed below:

```
case GameState.TitleScreen:
{
    if (titleTheme == null)
        titleTheme = soundBank.GetCue(titleThemeName);

    playTitleTheme();

    if (keyIsPressed(Keys.Space))
    {
        if (titleTheme.IsPlaying)
        {
```

```

        stopTitleTheme();
        audioEngine.GetCategory("Music").SetVolume
        (.5f);
    }

    exitCurrentState();
    changeState(GameState.Map);
}

break;
}

```

This segment was chosen for its length as a simple example of what might be within a given case of the `Update()` method. In this case, if the *currentState* variable's value is `GameState.TitleScreen` (an enumeration), then it enters the case block. First, it checks to see if the *titleTheme* variable exists, then gets the cue for it from the *soundBank* variable and begins playing the theme. While the title screen is up (until the user presses *Space Bar*), the case checks to see if the user has pressed the *Space Bar* key. When they do, it immediately stops the title theme, changes the volume of the music files in the *audioEngine* variable, and then exits the title screen state, changing to the `GameState.Map` state instead. Each case behaves similarly to this, but with more or less data depending on the case itself.

2.10.5: Game1.Draw()

Like the `Update()` method, the `Draw()` method contains roughly half of the overall game logic, and is set up nearly identically to the `Update()` method in terms of functionality. Both the *currentState* and *menuState* variables affect different portions of code within this method, and a *switch* statement is also used here just as in the former method. The only three methods within `Draw()` that do not involve the *switch* statement on the *currentState* variable are the *spriteBatch.Begin()* and *spriteBatch.End()* methods, as well as the *base.Draw(gameTime)* method call. Below is a sample case from the *switch* statement within the `Draw()` method:

```

case GameState.TitleScreen:
{
    graphics.GraphicsDevice.Clear(Color.Black);
    spriteBatch.Draw(title.spriteTexture, new Rectangle(0, 0,
    title.spriteTexture.Width, title.spriteTexture.Height),
    Color.White);

    spriteBatch.DrawString(titleFont, "Clouded Horizons:
    Planeswalker Chronicles", new Vector2(150, 150),
    Color.Black);
    spriteBatch.DrawString(menuFont, "Press Space to continue, or
    Esc to exit...", new Vector2(375, 400), Color.Black);

    break;
}

```

As above, this case was chosen as an example because of its short length. If the current game state is that of the title screen, then this case will be entered. The *graphics* variable (of type

GraphicsDeviceManager) will first call its `Clear()` method with the parameter *Color.Black*, which will clear the screen to a blank, black background. Then, the *spriteBatch* object will draw the title screen image to the screen, followed by drawing the strings for the title and the "Press Space to continue..." line below them at the specified locations.

Chapter III: Conclusion

3.1: Lessons

Debug often, scaling the number of debugging runs up with the difficulty and complexity of the pieces being added. I was saved many times by debugging after every new function and often during, when small issues would arise that could have easily built up over time.

Inheritance is nice, but there is such a thing as too much of a good thing. Initially, I used inheritance for many things that I didn't need to, such as individual weapons and armor. That was later removed once I found it easier to read from files than to set up classes for every item.

A little forethought can make life much easier when dealing with pieces that have multiple cases. My logic methods more often than not involved several different cases, sometimes numbering in the double digits for things like combat, and taking the time to think about each case before I began coding often saved me lots of time that would have otherwise been spent going back and implementing options and cases as I thought of them.

3.2: What Could Be Done Differently?

If I were to redo this project, I would like to tone down the complexity of the system somewhat. While I enjoyed the difficulty and the experience from implementing such a system, I would like to simplify the system somewhat so as to simplify implementation and method creation.

I would love to clean up a lot of my graphical code and make it both cleaner and easier to understand. When I began this project, I had no knowledge of XNA or any of its features or language, and so this was a project in which I learned how to do things at the same time as I did them. With the knowledge I have now, I believe that I could both handle the graphical aspects in a cleaner and more efficient fashion.

3.3: What Could Be Added?

It would be fairly simple for me to add more of the extraneous RPG elements such as additional classes, additional quests, more areas, etc., due to the method in which I set up my code's inheritance tree and data reading methods. I would also like to change the system so that I could add elements that are more unique over those which are more common in multiple RPGs.

3.4: Known Bugs or Glitches

There are some small issues with movement, wherein the character will not end up in the correct spot when going through some doors. There is also a small animation glitch where, if a CPU player goes twice in a row or CPU players move one after the other, only the last CPU's damage animation will fully finish.

3.5: Acknowledgments

This game was created solely for the purposes of education, and will not be used for financial gain. The character and player sprites used in this game were obtained from www.videogamesprites.net, while the other images associated with this project were drawn by me using a program called TileStudio, described in the first section of this documentation.

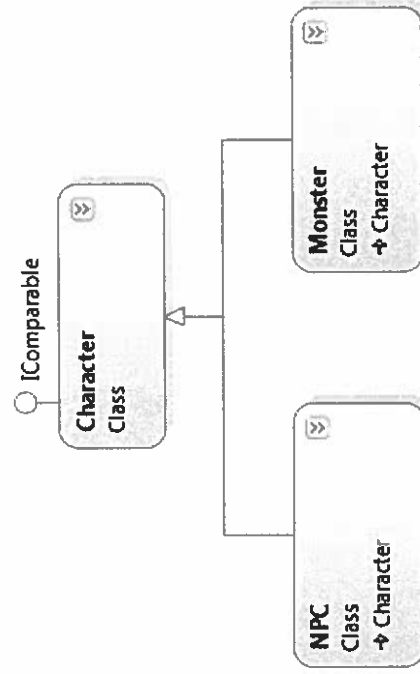
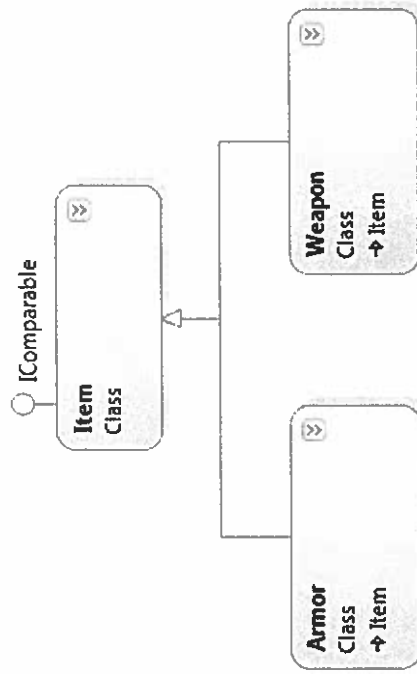
CLOUDED HORIZONS: PLANESWALKER CHRONICLES

Senior Defense
February 22, 2010
Benji Powell

Introduction

- ▣ What is an RPG?
- ▣ Why make one?
- ▣ What is XNA?

Inheritance



Job Class

Enchantment Class

Effect Class

Skill Class

Quest Class

MenuButton Class

Sprite Class

Tile Class

What Did Inheritance Help?

- ▣ Lots of reused code, especially when dealing with the combat system.
- ▣ Simple and logical to visualize, both in and out of code.
- ▣ Simpler to debug - exceptions became clearer; IE "unable to cast" exception versus an issue with a class function.

Inheritance Example

- ▣ Quest class - needed some way to have an object that had an overarching type, but logic specific to a particular quest.
- ▣ Answer - make a general Quest class, which had virtual methods passed down to the child quests (specific quest objects) which handled the quest-specific logic like checking the progress or handing out rewards.
- ▣ Without inheritance, would have had to rely on a massive case block for quests, conditions, rewards, etc.

Game Design

- ▣ Two types of RPGs – turn-based, real-time. This project is turn-based.
- ▣ State-based system – stack of game states that determine what logic should be used via the Update() method. Similar idea used in the combat system, but only a MenuState variable, not a stack.
- ▣ Graphics are 2-dimensional.

Difficulties

- ▣ Learning the software and language – no prior knowledge of C# or XNA.
- ▣ Debugging – lots of code, difficult loops, unfamiliar exceptions.
- ▣ Logic – knowing where to place what code for triggering at the correct time. (Happened a lot with the combat system and quests.)

What Would I Do Differently?

- ▣ Clean up a lot of my graphical code – much better understanding of how XNA handles graphics now versus at the beginning.
- ▣ Potentially change menu styles – works fine for my purposes, but there could be issues when dealing with classes with an excessive amounts of skills (too many buttons to display).
- ▣ Try and simplify some of the system mechanics so that the code is less complex.
- ▣ Set as much as I could up to be read from files at the outset.

Lessons

- ▣ Debug *constantly*. Minor-seeming changes can and will have unforeseen consequences in the code, especially with underlying loops such as those in the `Update()` and `Run()` methods.
- ▣ Inheritance is a good thing, but it doesn't have to be used for everything. Some things stand on their own perfectly fine.
- ▣ Take time to consider multiple-case ideas and segments carefully. Getting nearly everything at the start is much better than having to work in half the cases later down the line.